Structuri de Date

Laborator de recapitulare

Alexandru Tudor

21 aprilie 2026

1. Introducere

Scopul acestui laborator îl reprezintă **recapitularea** noțiunilor studiate în laboratoarele 01–08, prin exerciții de diferite tipuri:

- **Probleme aplicative** – folosirea unei structuri cunoscute pentru rezolvarea unei probleme concrete;
- **Bug-fixing** – depanarea unor implementări de structuri în care au fost introduse erori intenționate;
- **Complete-the-code** – implementarea unor funcții-lipsă într-un schelet deja existent;
- **Probleme care combină 2 sau mai multe structuri** – utilizarea diferitelor structuri de date.

2. Structura laboratorului

Laboratorul conține scheletul de cod pe care trebuie să îl completați/corectați. Fiecare exercițiu are un subdirector propriu cu un `Makefile` care expune regulile: `build`, `test`, `test-valgrind`, `clean`. La nivelul `skeleton/` există un `Makefile` agregat care rulează toate seturile de teste și afișează scorul total din 100 de puncte.

Compilare și testare:

```
$ cd skeleton
$ make build         # compileaza toate cele 6 exercitii
$ make test          # ruleaza testele si afiseaza scorul total
$ make test-valgrind # verifica pentru memory leaks
$ make clean         # sterge binarele
```

3. Cerințe

Cerința 1 (15p) (*Hot Potato* – problemă aplicativă, Queue)

N jucători, numerotați $1..N$, stau în cerc și își pasează un cartof „fierbinte”. Regulile jocului:

- Jucătorii **numără în cerc**: jucătorul care ține cartoful în acest moment este „numărul 1”; cel care îl primește după prima pasare este „numărul 2”; cel de după următoarea pasare este „numărul 3”; și așa mai departe.
- Al **k-lea jucător** (numărând de la începutul rundei) **este eliminat** din joc. Asta înseamnă că au loc exact $k-1$ pasări, iar jucătorul care ține cartoful după aceste pasări iese din joc.
- Jocul continuă cu jucătorul imediat următor (care devine „numărul 1” al noii runde) și procesul se repetă până rămâne **un singur jucător** – câștigătorul.

Exemple rapide pentru a fixa semantica lui k:

- $k = 1$: primul jucător al rundei este eliminat (zero pasări).
- $k = 2$: „numărul 1” pasează o dată, apoi „numărul 2” este eliminat (o pasare, apoi iese cel care a primit).
- $k = 3$: au loc 2 pasări; „numărul 3” este eliminat.

Modelarea cu o coadă

Cercul de jucători se modelează natural cu o coadă (`Queue.h`):

- **Front-ul cozii** = jucătorul care *ține în acest moment* cartoful.
- **A pasa** înseamnă: se scoate jucătorul din coadă și se pune înapoi la capătul cozii.
- **A elimina** înseamnă: se face `dequeue` fără a-l pune înapoi.

Exemplu

Pentru $N = 5, k = 2$, coada evoluează astfel:

Pas	**Coada (front → rear)**	**Acțiune**
Start	[1, 2, 3, 4, 5]	–
$k-1 = 1$ pasare	[2, 3, 4, 5, 1]	1 a pasat, a mers la capăt
Eliminare	[3, 4, 5, 1]	**2** iese din joc
$k-1 = 1$ pasare	[4, 5, 1, 3]	3 a pasat
Eliminare	[5, 1, 3]	**4** iese din joc
$k-1 = 1$ pasare	[1, 3, 5]	5 a pasat
Eliminare	[3, 5]	**1** iese din joc
$k-1 = 1$ pasare	[5, 3]	3 a pasat
Eliminare	[3]	**5** iese din joc

Rămâne jucătorul **3** – câștigătorul. Ordinea eliminărilor este $2, 4, 1, 5$.

Semnătura funcției

În fișierul `hot_potato.c` implementați:

```
int hotPotato(int N, int k, int *eliminationOrder);
```

- `N` – numărul de jucători ($N \geq 1$);
- `k` – valoarea până la care se numără; al k-lea jucător, contând de la începutul rundei, este eliminat ($k \geq 1$);
- `eliminationOrder` – vector de dimensiune cel puțin $N - 1$ în care se scriu jucătorii eliminați, în ordinea ieșirii din joc;
- **returnează** id-ul câștigătorului.

Constrângeri:

- Implementarea folosește **obligatoriu** o coadă (`Queue.h`).
- Nu folosiți recursivitate și nu înlocuiți coada cu un vector auxiliar.
- Cazuri limită: $N = 1$ (un singur jucător $\Rightarrow$ el este câștigătorul, nu se elimină nimeni), $k = 1$ (fiecare jucător iese imediat ce primește cartoful – fără pasări intermediare).

Alte exemple:

Apel	**Câștigător**	**Ordinea eliminărilor**
`hotPotato(5, 2, ...)`	3	2, 4, 1, 5
`hotPotato(5, 1, ...)`	5	1, 2, 3, 4
`hotPotato(7, 3, ...)`	4	3, 6, 2, 7, 5, 1

Cerința 2 (15p) (*Evaluator postfix* – problemă aplicativă, Stack)

În **notația postfixată** (sau *reverse Polish*), operatorii sunt scriși *după* operanzii lor, spre deosebire de notația uzuală (infixată) în care operatorul stă între operanzi.

Exemple de corespondență infix ↔ postfix:

Infix	**Postfix**	**Valoare**
`3 + 4`	`3 4 +`	7
`(12 + 3) * 5`	`12 3 + 5 *`	75
`5 + (1 + 2) * 4 - 3`	`5 1 2 + 4 * + 3 -`	14

Avantajul notației postfixate este că **nu are nevoie de paranteze**: ordinea operatorilor este dată complet de poziție. Ea se evaluează foarte natural cu o stivă.

Modelarea folosind stivă

Expresia se parcurge token cu token (token-urile sunt separate prin spații). Pentru fiecare token:

- Dacă este un **număr**, se pune pe stivă (`push`).
- Dacă este un **operator** ($+, -, *, /$): se scot de pe stivă cei doi operanzi (`pop` de două ori), se aplică operația, iar rezultatul se pune înapoi pe stivă.

La finalul parcurgerii, pe stivă trebuie să rămână exact **un singur element** – valoarea expresiei.

Atenție la ordinea operanzilor: dacă operatorul nu este comutativ ($-$ sau $/$), ordinea contează. Dacă din stivă se extrag pe rând `b` (primul scos, era deasupra) și apoi `a` (al doilea scos), rezultatul este `a op b`, nu `b op a`.

Exemplu

Pentru expresia `"12 3 + 5 *"`:

Token	**Stiva (jos → sus)**	**Acțiune**
—	[]	stiva inițial vidă
`12`	[12]	`push(12)`
`3`	[12, 3]	`push(3)`
`+`	[15]	`pop` → 3, `pop` → 12; `push(12+3)`
`5`	[15, 5]	`push(5)`
`*`	[75]	`pop` → 5, `pop` → 15; `push(15*5)`

Stiva conține un singur element la final, deci rezultatul este **75**.

Cazuri de eroare (validare).

Expresia este **invalidă** dacă apare oricare din:

- un operator când pe stivă sunt mai puțin de 2 operanzi (ex: `"3 +"`);
- la final, pe stivă rămân 0 sau mai multe elemente decât 1 (ex: `"1 2 3 +"`);
- apare un caracter necunoscut (ex: `"3 4 x"`);
- împărțire prin 0 (ex: `"10 0 /"`);
- expresia este șirul vid.

În orice caz de eroare, funcția returnează constanta `LONG_MIN` (definită în `<limits.h>`).

Semnătura funcției

În fișierul `postfix.c` implementați:

```
long evaluatePostfix(const char *expr);
```

- `expr` – șir de caractere conținând o expresie postfixată cu operanzi întregi și operatori din $\{+, -, *, /\}$, separați prin spații;
- **returnează** valoarea expresiei, sau `LONG_MIN` dacă expresia este invalidă.

Constrângeri:

- Implementarea folosește **obligatoriu** o stivă (`Stack.h`).
- Operanzii pot avea **mai multe cifre** (ex: `"100 200 +"`).
- Folosiți tipul `long` (nu `int`) pentru valori, pentru a permite rezultate mari.

Alte exemple:

Input	Output
`"3 4 +"`	7
`"12 3 + 5 *"`	75
`"5 1 2 + 4 * + 3 -"`	14
`"100 200 +"`	300
`"42"`	42
`"+"`	`LONG_MIN`
`"1 2 3 +"`	`LONG_MIN`
`"10 0 /"`	`LONG_MIN`
`"3 4 x"`	`LONG_MIN`

Cerința 3 (25p) (*Bug-fixing* – OrderedSet + BinaryTree + AVL)

Aveți la dispoziție implementări pentru trei structuri studiate anterior: **OrderedSet**, **arbore binar** și **arbore AVL**. Fiecare implementare conține unul sau mai multe bug-uri introduse intenționat, care fac ca anumite teste să pice.

Găsiți bug-urile și corectați-le astfel încât toate testele să treacă. Restul codului funcționează corect – nu rescrieți modulele de la zero.

Distribuția bug-urilor

Fișier	Bug-uri introduse
`OrderedSet.h`	2
`bintree.c`	1
`avl.c`	1

Corectați cele 4 bug-uri astfel încât:

```
$ make test
...
SCOR BugFix: 25.00 / 25.00
```

Nu modificați fișierul de teste (`testEx3.c`) și **nu schimbați** semnătura funcțiilor. Doar corpul funcțiilor din cele 3 fișiere trebuie modificat.

Cerința 4 (15p) (*BST utils* – complete-the-code, BST)

Într-un **arbore binar de căutare (BST)**, fiecare nod respectă proprietatea:

- toate valorile din subarborele stâng sunt **strict mai mici** decât valoarea nodului;
- toate valorile din subarborele drept sunt **strict mai mari** decât valoarea nodului.

Trebuie să implementați două funcții utilitare pe BST:

```
int isBST(Tree root);
int kthSmallest(Tree root, int k);
```

Partea I: `isBST(root)`

Returnează 1 dacă arborele primit respectă proprietatea BST și 0 altfel. Arborele vid este considerat BST valid. Atenție: un test *local* (copilul stâng < părinte < copilul drept) **nu este suficient**. Exemplu de arbore care „păcălește" un test local:

```
      10
     /  \
    5    15
   / \
  3   12
```

La nodul 5, copiii (3, 12) respectă $3 < 5 < 12$ – local arată corect. Dar 12 se află în subarborele stâng al rădăcinii 10, deci ar trebui să fie < 10 – **proprietatea BST este încălcată**.

Partea II: `kthSmallest(root, k)`

Returnează al k-lea cel mai mic element din BST (indexat de la 1). Pentru k invalid ($k < 1$ sau $k >$ numărul de noduri) sau arbore vid, returnează `INT_MIN`.

Exemplu

Pentru arborele:

```
      5
     /  \
    3    7
   / \
  1   4
```

Apel	Rezultat	Observație
`kthSmallest(root, 1)`	1	cel mai mic
`kthSmallest(root, 3)`	4	al treilea
`kthSmallest(root, 5)`	7	cel mai mare
`kthSmallest(root, 0)`	`INT_MIN`	k invalid
`kthSmallest(root, 6)`	`INT_MIN`	$k >$ size

Constrângeri:

- Nu modificați `tree.h` sau primitivele din `tree.c` – le puteți folosi, dar nu le schimbați.
- Nu construiți vectori auxiliari cu toate valorile din arbore – ambele funcții trebuie implementate folosind direct structura arborelui.

Cerința 5 (15p) (*Swap Pairs în DLL* – complete-the-code, DoubleLinkedList)

Primiți o listă dublu-înlănțuită și trebuie să **interschimbați fiecare pereche de noduri adiacente** (nodurile de pe pozițiile 0-1, 2-3, 4-5, ...).

Exemple:

Înainte	**După**
`[1, 2, 3, 4, 5, 6]`	`[2, 1, 4, 3, 6, 5]`
`[1, 2, 3, 4, 5]`	`[2, 1, 4, 3, 5]` (ultimul rămâne singur)
`[1, 2]`	`[2, 1]`
`[1]`	`[1]` (nu se schimbă nimic)

Operația se face **in-place**, fără alocări noi de noduri. Dacă lista are lungime impară, ultimul nod rămâne pe poziția inițială.

Exemplu

Pentru `[A, B, C, D]`, evoluția pointerilor:

Pas	**Starea listei (front → rear)**
Start	`A` ↔ `B` ↔ `C` ↔ `D`
Swap (A, B)	`B` ↔ `A` ↔ `C` ↔ `D`
Swap (C, D)	`B` ↔ `A` ↔ `D` ↔ `C`
Update last	`B` ↔ `A` ↔ `D` ↔ `C`

Semnătura funcției

În fișierul `swap_pairs.h` implementați:

```
void swapPairs(List *list);
```

- `list` – pointer la lista dublu-înlănțuită (`List*`). Se modifică in-place.
- Pentru `list` `NULL`, lista vidă sau listă cu un singur nod – funcția nu face nimic.

Constrângeri:

- **In-place**: nu alocați noduri noi, nu folosiți un vector auxiliar sau alte structuri.
- Nu schimbați valorile nodurilor (`->elem`) – rearanjați nodurile.
- După operație, `list->first`, `list->last` și toate legăturile `prev`/`next` trebuie să fie **coerente** – testele verifică explicit parcurgerea bidirecțională.
- Un apel dublu succesiv (`swapPairs(l); swapPairs(l);`) trebuie să readucă lista la forma inițială.

Cerința 6 (15p) (*Merge K liste sortate* – combinat, SortedList + Heap)

Aveți un vector de K liste simplu-înlănțuite, fiecare sortată crescător. Trebuie să construiți **o listă nouă, sortată crescător**, care conține toate elementele din cele K liste, folosind un **min-heap** de dimensiune cel mult K.

Exemplu:

Intrare (K=3)	**Ieșire**
`L1 = [1, 5]`	
`L2 = [2, 3]`	`[1, 2, 3, 4, 5]`
`L3 = [4]`	

Se folosesc *două structuri*:

- `SortedList` – pentru listele de intrare și lista-rezultat.
- `Heap` – pentru a alege eficient *următorul minim* dintre capetele celor K liste.

Ideea algoritmică

Minimul global al celor K liste este, la orice moment, *unul dintre capetele* celor K liste (fiecare listă fiind sortată, capătul este cel mai mic element al ei). Folosim heap-ul să reținem mereu cele (cel mult) K capete curente.

1. Introducem în heap primul nod din fiecare listă ne-nulă.
2. Extragem minimul din heap $\Rightarrow$ acesta este următorul element al listei-rezultat. Adăugăm *un nod nou* (copie a valorii) la finalul listei-rezultat.
3. Dacă nodul extras avea un succesor în lista lui de origine, îl introducem în heap.
4. Repetăm până când heap-ul devine gol.

Trick: max-heap folosit ca min-heap

Heap-ul din `Heap.h` este un **max-heap** – extrage elementul cu prioritatea *cea mai mare*. Pentru a-l folosi ca min-heap, nu modificăm structura; în schimb, stocăm prioritatea **negată**:

- La `heapInsert`: `prior = -value`.
- La `getMax`: valoarea reală a minimului se obține inversând semnul (`-top.prior`), sau direct din `top.data->value`.

În câmpul `data` al heap-ului reținem **pointer-ul la nodul din lista originală** – ca să știm, după ce am extras minimul, de unde să luăm succesorul.

Exemplu

Pentru exemplul de mai sus (`L1=[1,5]`, `L2=[2,3]`, `L3=[4]`):

Pas	Heap (valori reale)	Extras	Rezultat
Start (inițializare)	{1, 2, 4}	–	`[]`
Scot 1 (din L1), push 5	{2, 4, 5}	1	`[1]`
Scot 2 (din L2), push 3	{3, 4, 5}	2	`[1, 2]`
Scot 3 (din L2), L2 gol	{4, 5}	3	`[1, 2, 3]`
Scot 4 (din L3), L3 gol	{5}	4	`[1, 2, 3, 4]`
Scot 5 (din L1), L1 gol	{}	5	`[1, 2, 3, 4, 5]`

Rezultat: `[1, 2, 3, 4, 5]`. Heap-ul n-a depășit niciodată dimensiunea 3 ($= K$).

Semnătura funcției

În fișierul `merge_k.h` implementați:

```
SortedList mergeKSortedLists(SortedList *lists, int K);
```

- `lists` – vector de K liste sortate crescător; fiecare `lists[i]` poate fi `NULL`;
- `K` – numărul de liste ($K \geq 0$);
- **returnează** o listă nouă, sortată crescător, cu toate elementele. Returnează `NULL` dacă `K == 0`, `lists == NULL`, sau toate listele sunt `NULL`.

Constrângeri:

- Folosiți **obligatoriu** un min-heap (realizat prin negarea priorităților în heap-ul max).
- Listele de intrare **nu trebuie modificate** – pentru lista-rezultat alocați *noduri noi*, nu refolosiți nodurile din liste.
- La final, heap-ul trebuie eliberat cu `freeQueue`.

4. Barem și punctaj

Fiecare exercițiu are propriile teste, iar scorul total este 100p.

Cerința	Tip	Punctaj
1. Hot Potato	aplicativ	15.00
2. Evaluator postfix	aplicativ	15.00
3. Bug-fixing (3 module)	depanare	25.00
4. isBST + kthSmallest	complete-the-code	15.00
5. Swap Pairs DLL	complete-the-code	15.00
6. Merge K liste sortate	combinat (2 structuri)	15.00
Total		**100.00**

Pentru o rulare completă, `make test` de la nivelul directorului de exercițiu afișează:

```
$ make test
single_player       - Toate testele au trecut!  Puncte: 2.00
k_one               - Toate testele au trecut!  Puncte: 2.00
classic             - Toate testele au trecut!  Puncte: 3.00
seven_three         - Toate testele au trecut!  Puncte: 3.00
null_elim           - Toate testele au trecut!  Puncte: 2.00
large               - Toate testele au trecut!  Puncte: 3.00

Teste rulate: 6 / 6
SCOR HotPotato: 15.00 / 15.00
```

Iar la nivelul directorului `skeleton/`, `make test` agregă scorul celor șase exerciții:

```
$ make test
...
========================================
SCOR TOTAL: 100.00 / 100.00
========================================
```

5. Observații finale

- Ne așteptăm ca implementările voastre să treacă valgrind (verificat cu `make test-valgrind`).
- Compilările folosesc `gcc` cu `-std=c11 -Wall -Wextra`. Warning-urile sunt considerate erori – corectați-le.
- **Success!**